Backend متقدم: ملخص الجلسات

Session + Refresh Token · Index · Caching · Transaction (NestJS + Prisma + PostgreSQL)

ترتيب الجلسات

#	الموضوع	المدة
1	Session + Refresh Token: Login و Refresh و Rotation	60 د
2	Logout و الأجهزة و logout-others	60 د
3	Index + EXPLAIN ANALYZE	60 د
4	Caching	60 د
5	Transaction + Race Condition	60 د

شكل كل جلسة (نفس الخطوات دائماً)

1. **المشكلة** اشرحها بدقيقتين بدون كود.
2. **الكود الغلط** اكتبه قدامهن، و خليهن يشوفوا العطل بأرقام.
3. **الحل** اكتبه معهن خطوة خطوة.
4. **التجربة** يعيدوا نفس الاختبار و يقارنوا.
5. **نقاش** سؤال أو تحدّي صغير بآخر الجلسة.

القاعدة: لا تشرح الحل قبل ما يشوفوا المشكلة بإيديهن. الأرقام بتقنعهن أكتر من الكلام.

1. Session + Refresh Token + Session ID

المشكلة

عندهن JWT واحد صلاحيته طويلة. شو بيصير لو:

- المستخدم عمل Logout؟ التوكن لسا شغال.
- توكن انسرق؟ المهاجم عنده صلاحية لحد ما ينتهي.
- بدنا نخرجه من جهاز واحد بس؟ مستحيل.

الفكرة

- **Access Token:** قصير (15 دقيقة)، بيحمل `sub` و `role` و `sid`.
- **Refresh Token:** طويل (7 أيام)، بيستخدم بس لطلب Access جديد.
- **Session ID (`sid`):** سجل بقاعدة البيانات. كل جهاز عنده Session خاصة فيه.

Prisma

```
model Session {
  id               String    @id @default(uuid())
  userId           Int
  user             User      @relation(fields: [userId], references: [id])
  refreshTokenHash String
  userAgent        String?
  expiresAt        DateTime
  revokedAt        DateTime?
  createdAt        DateTime  @default(now())

  @@index([userId])
}
```

ما منخزّن الـ Refresh Token نفسه بقاعدة البيانات. منخزّن **Hash** منه، مثل كلمة المرور.

1. Session: Login

الحل Login ينشئ Session

```
async login(dto: LoginDto, userAgent?: string) {
  const user = await this.validateUser(dto.email, dto.password);

  const session = await this.prisma.session.create({
    data: {
      userId: user.id,
      userAgent,
      refreshTokenHash: '',
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000),
    },
  });

  const tokens = await this.signTokens(user, session.id);
  await this.saveRefreshHash(session.id, tokens.refreshToken);
  return tokens;
}
```

توقيع التوكنات

```
private async signTokens(user: { id: number; role: string }, sid: string) {
  const [accessToken, refreshToken] = await Promise.all([
    this.jwt.signAsync(
      { sub: user.id, role: user.role, sid },
      { secret: process.env.JWT_ACCESS_SECRET, expiresIn: '15m' },
    ),
    this.jwt.signAsync(
      { sub: user.id, sid },
      { secret: process.env.JWT_REFRESH_SECRET, expiresIn: '7d' },
    ),
  ]);
  return { accessToken, refreshToken };
}
```

حفظ الـ Hash

```
import { createHash } from 'crypto';

private hash(token: string) {
  return createHash('sha256').update(token).digest('hex');
}

private saveRefreshHash(sid: string, token: string) {
  return this.prisma.session.update({
    where: { id: sid },
    data: { refreshTokenHash: this.hash(token) },
  });
}
```

ليش `sha256` مو `bcrypt`؟ لأن bcrypt بيقطع المدخل عند 72 بايت، و الـ JWT أطول من هيك.

1. Session: Refresh مع Rotation

الحل

```
async refresh(refreshToken: string) {
  let payload: { sub: number; sid: string };
  try {
    payload = await this.jwt.verifyAsync(refreshToken, {
      secret: process.env.JWT_REFRESH_SECRET,
    });
  } catch {
    throw new UnauthorizedException();
  }

  const session = await this.prisma.session.findUnique({
    where: { id: payload.sid },
  });

  if (!session || session.revokedAt || session.expiresAt < new Date()) {
    throw new UnauthorizedException();
  }

  // توكن قديم انستخدم مرة تانية = مشبوه، منلغي الجلسة
  if (session.refreshTokenHash !== this.hash(refreshToken)) {
    await this.prisma.session.update({
      where: { id: session.id },
      data: { revokedAt: new Date() },
    });
    throw new UnauthorizedException();
  }

  const user = await this.prisma.user.findUniqueOrThrow({
    where: { id: payload.sub },
  });
  const tokens = await this.signTokens(user, session.id);
  await this.saveRefreshHash(session.id, tokens.refreshToken);
  return tokens;
}
```

نقاط للشرح

- **Rotation:** كل Refresh بيعطي Refresh Token جديد، و القديم بيبطل.
- **Reuse Detection:** إذا انستخدم توكن قديم، يعني في نسخة منه عند حدا غير صاحبه. منلغي الجلسة كلها.

1. Session: Logout و الأجهزة

الحل

```
// جهاز واحد
logout(sid: string) {
  return this.prisma.session.update({
    where: { id: sid },
    data: { revokedAt: new Date() },
  });
}

// كل الأجهزة ما عدا الحالي
logoutOthers(userId: number, currentSid: string) {
  return this.prisma.session.updateMany({
    where: { userId, id: { not: currentSid }, revokedAt: null },
    data: { revokedAt: new Date() },
  });
}
```

و endpoint اسمه `GET /auth/sessions` بيرجّع الأجهزة المسجّلة (`userAgent` و `createdAt`) مع زر "اخرج من هالجهاز".

التجربة (Postman)

1. Login مرتين و غيّر الـ header `User-Agent` ("جهازين").
2. اعمل `refresh` على جهاز 1 و شوف التوكن الجديد.
3. جرّب الـ Refresh Token القديم. لازم يعطي 401.
4. اعمل `logout-others` من جهاز 1، وبعدين جرّب refresh على جهاز 2.

سؤال للنقاش

الـ Access Token يضل صالح 15 دقيقة بعد الـ Logout. كيف منحلها؟
الحل: الـ Guard يتحقق من الـ Session بقاعدة البيانات أو Redis. المقابل: طلب إضافي بكل Request. هاد درس Trade-offs بحد ذاته.

2. Index

المشكلة

جدول فيه مليون كورس، و البحث بالعنوان بطيء.

الإعداد

```
INSERT INTO "Course" ("title")
SELECT 'Course ' || n FROM generate_series(1, 1000000) AS n;
```

عدّل حسب الحقول الإجبارية بالـ schema تبعك (مثل `teacherId`).

قبل

```
EXPLAIN ANALYZE
SELECT * FROM "Course" WHERE title = 'Course 999999';
```

خليهن يلاحظوا `Seq Scan` و `Execution Time`.

الحل

schema.prisma

```
model Course {
  id    Int    @id @default(autoincrement())
  title String

  @@index([title])
}
```

```
npx prisma migrate dev --name add_course_title_index

-- بعدها بـ psql:
ANALYZE "Course";
```

أعد نفس الاستعلام. لازم يظهر `Index Scan` و الزمن ينزل بشكل واضح.

3 أشياء مهمة

1. **الـ Foreign Keys** ما بتتفهرس تلقائياً بـ PostgreSQL. `teacherId` بالـ where أو الـ join بده `@@index([teacherId])`.
2. **البحث بـ** `ILIKE '%text%'` (**contains**) ما بيستفيد من B-Tree عادي. جرّبوه و شوفوا الخطة.
3. **الـ Index مو مجاني:** بيبطّئ INSERT و UPDATE و بياخد مساحة. جرّب إدخال 100 ألف سجل مع Index و بدونه.

تحدّي

`WHERE "teacherId" = 5 ORDER BY title`: وين بتحط الـ Index؟ جرّب `@@index([teacherId, title])` و قارن.

3. Caching

المشكلة

`GET /courses/:id` بيروح على PostgreSQL كل مرة، حتى لو 1000 مستخدم طلبوا نفس الكورس.

الإعداد

```
npm i @nestjs/cache-manager cache-manager

// app.module.ts
CacheModule.register({ isGlobal: true, ttl: 10_000 }) // 10 ثواني (ms بالإصدارات الحديثة)
```

قبل

`findOne` بس بينفّذ `this.prisma.course.findUnique(...)` بدون أي تخزين.

الحل (Cache-Aside)

```
constructor(
  private prisma: PrismaService,
  @Inject(CACHE_MANAGER) private cache: Cache,
) {}

async findOne(id: number) {
  const key = `course:${id}`;

  const cached = await this.cache.get(key);
  if (cached) {
    console.log('CACHE HIT');
    return cached;
  }

  console.log('CACHE MISS');
  const course = await this.prisma.course.findUnique({ where: { id } });
  if (course) await this.cache.set(key, course);
  return course;
}

async update(id: number, dto: UpdateCourseDto) {
  const course = await this.prisma.course.update({ where: { id }, data: dto });
  await this.cache.del(`course:${id}`); // Invalidation
  return course;
}
```

التجربة

1. طلبين متتاليين: Miss ثم Hit.
2. عدّل الكورس **بدون** `cache.del` و شوف البيانات القديمة (Stale Data)، وبعدين أضفه و شوف الفرق.
3. قِس الزمن بـ Postman قبل و بعد.

نقاش

- ليش منخزّن الكورس بالـ id و ما منخزّن قوائم الـ pagination؟ عدد المفاتيح كبير (page × limit × search) و إبطالها صعب. القوائم بنخزّنها بـ TTL قصير بس.
- ما تحط بيانات خاصة بكل مستخدم بمفتاح مشترك. و الـ In-Memory Cache خاص بكل نسخة من التطبيق (الحل لاحقاً: Redis).

4. Transaction + Race Condition

المشكلة

كورس فيه مقعد واحد، و 10 طلاب بيسجلوا بنفس اللحظة.

Prisma

```
model Enrollment {
  id       Int @id @default(autoincrement())
  userId   Int
  courseId Int

  @@unique([userId, courseId])
}

model Course {
  // ...
  seats Int @default(1)
}
```

الكود الغلط

```
async enroll(userId: number, courseId: number) {
  const course = await this.prisma.course.findUniqueOrThrow({
    where: { id: courseId },
  });
  if (course.seats <= 0) throw new ConflictException('No seats left');

  await this.prisma.course.update({
    where: { id: courseId },
    data: { seats: { decrement: 1 } },
  });
  return this.prisma.enrollment.create({ data: { userId, courseId } });
}
```

سبب المشكلة

```
A: read seats = 1
B: read seats = 1
A: decrement, seats = 0
B: decrement, seats = -1
```

4. Transaction: الاختبار و الحل

سكربت الاختبار (race.js)

```
const run = async (i) => {
  const res = await fetch('http://localhost:3000/courses/1/enroll', {
    method: 'POST',
    headers: { Authorization: `Bearer ${TOKENS[i]}` },
  });
  return res.status;
};
Promise.all(TOKENS.map((_, i) => run(i))).then(console.log);
```

`TOKENS` = توكنات 10 مستخدمين. مع الكود الغلط: أكتر من واحد بيسجل، و المقاعد بتصير سالب.

الحل

```
async enroll(userId: number, courseId: number) {
  try {
    return await this.prisma.$transaction(async (tx) => {
      const result = await tx.course.updateMany({
        where: { id: courseId, seats: { gt: 0 } },
        data: { seats: { decrement: 1 } },
      });

      if (result.count === 0) {
        throw new ConflictException('No seats left');
      }

      return tx.enrollment.create({ data: { userId, courseId } });
    });
  } catch (e) {
    if (e instanceof Prisma.PrismaClientKnownRequestError && e.code === 'P2002') {
      throw new ConflictException('Already enrolled');
    }
    throw e;
  }
}
```

نقاط للشرح

- الشرط (`seats > 0`) و التنقيص بعملية وحدة ذرية داخل قاعدة البيانات. مو `if` بالكود.
- لو فشل الـ `create` ، كل شي بيرجع (Rollback).
- `@@unique([userId, courseId])` بيمنع نفس الطالب يسجّل مرتين. الخطأ `P2002` منحوّله لـ `ConflictException` .

النتيجة المتوقعة

أعد السكربت: واحد بس ينجح (201) و الباقي 409، و المقاعد تبقى 0.